

University of Pisa

Data Science & Business Informatics

DM 1 - Data Mining Project

Board Games Dataset

Marco Redaelli

Sara Biavasco

AY 2025-2026

Contents

1 Introduction **3**

2 Data Understanding and Preparation **3**

2.1 Data Semantics . . . 3

2.2 DType Analysis . . . 3

2.3 Distribution and Statistics . . . 4

2.4 Assessing Data Quality . . . 4

2.4.1 Missing Values . . . 4

2.4.2 Semantic inconsistencies and Wrong Data . . . 5

2.4.3 Outlier Detection . . . 6

2.5 Pairwise correlation . . . 6

2.6 Data transformation: Feature Engineering and Elimination of Attributes . . . 7

3 Clustering **8**

3.1 Analysis by centroid-based methods . . . 9

3.1.1 Parameter selection and application: . . . 9

3.1.2 Results: . . . 9

3.2 Analysis by density-based methods . . . 10

3.2.1 Parameter selection and application: . . . 10

3.2.2 Results: . . . 11

3.3 Analysis by hierarchical clustering . . . 11

3.3.1 Parameter selection and application: . . . 11

3.3.2 Results: . . . 12

3.4 Semantic Clustering . . . 12

3.4.1 Parameter selection and application: . . . 12

3.4.2 Results: . . . 13

3.5 Final Discussion . . . 14

4 Classification **15**

4.1 KNN . . . 15

4.1.1 Parameter selection and application: . . . 15

4.1.2 Results: . . . 15

4.2 Naïve Bayes . . . 16

4.2.1 Parameter selection and application: . . . 16

4.2.2 Results: . . . 16

4.3 Decision Trees . . . 17

4.3.1 Parameters selection and application: . . . 17

4.3.2 Results: . . . 18

4.4 Final Discussion . . . 18

5 Regression **19**

5.1 Simple Regression . . . 19

5.2 Multiple Regression 20
5.3 Final discussion 20

6 Pattern Mining **21**
6.1 Frequent pattern extraction 21
6.2 Association Rules 22
6.3 Rule Based Classifier 23

1 Introduction

This report documents the work completed in the Data Mining 1 project, the main objective of which is the analysis of a Board Games dataset that includes information about board games and their statistics, which are generated by an online board gaming community, BoardGameGeek.com. The main goal of this analysis is to identify hidden patterns in the data through both unsupervised and supervised learning techniques. A structured analytical process has been undertaken to derive meaningful insights from the data.

2 Data Understanding and Preparation

2.1 Data Semantics

The Board Games dataset is composed of 21925 instances and each is described by 46 attributes. We started by classifying the attributes into standard categories, as each attribute type determines which statistical operations are meaningful and which preprocessing steps are appropriate.

We first distinguished between categorical (qualitative) and numerical (quantitative) attributes.

Within Categorical attributes, nominal attributes take values from a finite set of categories with no inherent ordering, meaning that only equality comparisons are meaningful. Examples in our dataset include **BGGId**, **Name**, **Description**, **ImagePath**, and **Family**. Although **BGGId** is numeric and sequential, it was treated as a nominal attribute, as it serves solely as an identifier linking to the corresponding BoardGameGeek page.

Ordinal attributes also have a finite domain, but unlike nominal attributes, their values can be meaningfully ordered. Attributes such as **GameWeight** and **ComWeight** fall into this category (1–5 scale), along with **Rating**, **LanguageEase**, and **Rank:***.

Binary attributes, that in our case are asymmetric, contain only two possible values (0/1). Examples include **IsReimplementation**, **Kickstarted** and **Cat:***.

Every other attribute falls into the Numerical category, representing measurable quantities. The overall classification, strengthened by a deep understanding of the data, both semantically and quantitatively, guided subsequent decisions on encoding strategies, data type optimization, and distribution analysis throughout the preprocessing pipeline.

2.2 DType Analysis

We proceeded by verifying whether the actual storage data types were consistent with these semantic categories and optimized for memory efficiency. To this end, we leveraged the pandas data types, which provide greater semantic expressiveness and flexibility.

We therefore created a function that, for each column, checked whether it was of numeric or object type, and we applied the following validations:

For numeric variables:

- if a column contains only 0s and 1s, it corresponds to a Boolean attribute and can be stored as **dtype boolean**;
- if it is of type float but contains only values of the form .0 (i.e., without an actual decimal part), it can be safely converted to **Int***;
- if it contains only positive values, it may be stored as **UInt***;

For each resulting Int, UInt, or Float column, we checked the minimum and maximum values in order to downcast it to the most memory-efficient corresponding dtype (e.g., column YearPublished from int64 to Int16).

For object columns:

- if the column is composed exclusively of strings, it can be converted to **string dtype**, facilitating subsequent operations such as character normalization (lowercasing, stripping, and collapsing multiple spaces into a single one). It also allows us to clearly distinguish text attributes from other object-type columns, such as lists;

We applied these transformations, obtaining that columns were converted from 30.22 MB to 24.01 MB, resulting in a memory reduction of **20.56%**.

2.3 Distribution and Statistics

To analyze the general statistics of the dataset, we first verified, excluding the ID column, that no fully duplicated rows were present. We also implemented a more advanced validation procedure to detect partially duplicated records. Specifically, we evaluated whether identical rows existed differentiated only by 1 to 3 columns, to detect small variations between records.

For numerical attributes, we performed an exploratory analysis using the native pandas `df.describe()` function. This showed the mean, standard deviation, minimum, maximum, and quartiles of each attribute. We then visualized their distributions using bar plots, focusing in particular on the behaviour around the first and fourth quartiles. Additionally, box plots and scatter plots were used to gain a clearer understanding of the distributional shape and the presence of potential outliers.

While we created specific visualizations for each numerical attribute, this approach was not applicable to non-numerical ones. For categorical or nominal attributes, we examined the presence of unique or repeated values within each column. Unexpected duplicates were identified in the Name, Description, and ImagePath columns.

- **Name**: 369 duplicated values, with the same name appeared up to six times. For the majority of these cases (363 out of 369), the duplicated names corresponded to different publication years. The remaining six cases were manually inspected and confirmed not to represent the same game.
- **Description**: 15 duplicated values. We hypothesized that these entries referred to different versions of the same game. To validate this assumption, we performed entity resolution (ER), observing that 73.3% of the groups shared a common name prefix separated by a delimiter (e.g., ":" or "–"), 6.7% differed only by a numeric suffix, and 33.3% contained the terms "version" or "edition" in all but at most one entry. Overall, 93.3% of the groups satisfied at least one of these conditions. The remaining 6.7% corresponded to a single case in which the name differed only by the addition of the term "more".
- **ImagePath**: 4 duplicated values, indicating that the same image was associated with different implementations of a game.

2.4 Assessing Data Quality

Following the preliminary exploratory analysis, a systematic assessment of data quality was conducted in preparation for the overall analysis. In particular, we focused on the identification of missing values, incorrect data, and potential outliers.

2.4.1 Missing Values

The following attributes were found to contain missing values:

- **Family** exhibited the highest number of missing values, with 15,262 NaNs (approximately 70% of the dataset). The attribute was temporarily retained to support semantic clustering analyses.
- **LanguageEase** contained 5,891 missing values (approximately 27%). Moreover, the available values did not convey meaningful information, as they did not conform to the expected domain. Consequently, this attribute could not be reliably imputed.
- **ComAgeRec** presented 5,530 missing values (about 25% of the dataset). This attribute was retained only temporarily, as it was required to support a set of adjustments applied to other related features.
- **ImagePath** included only 17 missing values (approximately 0.08%). However, since this attribute did not contribute meaningfully to the analysis, no imputation was performed.
- **Description** contained a single missing value, since this was not directly used in the analysis, this missing value was not addressed.

2.4.2 Semantic inconsistencies and Wrong Data

[As part of the data cleaning process, we employed web scraping as one of the correction methods. Since the dataset represents a point-in-time snapshot, our interventions were limited to the correction of static attributes (immutable values explicitly reported on the BoardGameGeek box) verifying whether previously incomplete fields had been updated on bgg.com. We deliberately avoided attributes such as GameWeight or others, as doing so could introduce semantic inconsistencies with related variables (NumWeighVotes) and other bias!]

In the **YearPublished** attribute a substantial number of rows contained the value 0, which is not semantically meaningful for a publication year. We attempted to recover the correct publication years by scraping the BoardGameGeek website, using each game's BGGId as a reference. Through this process, we successfully corrected 55 out of the 193 problematic entries; the remaining 138 cases could not be recovered reliably and since this attribute represents a calendar year, imputing values such as the mean or median would have introduced bias. Therefore, we replaced all unresolved zeros with NaN, preserving the integrity of the variable.

Issues involving **MinPlayers** and **MaxPlayers** were detected due to missing, inconsistent, or logically impossible values. Three anomaly patterns were identified (when available, official values were recovered via scraping; otherwise, we enforced logical consistency through deterministic rules, avoiding mean/median imputations):

- **MinPlayers > MaxPlayers**: we treated MaxPlayers as erroneous and set MaxPlayers = MinPlayers.
- **MinPlayers = 0 and MaxPlayers > 0**: we imputed MinPlayers = 1, since 0 players is not semantically meaningful and 1 is the most conservative valid choice.
- **MinPlayers = 0 and MaxPlayers = 0**: both values were set to NaN, marking them as missing.

On 182 problematic rows identified only 36 were set to NaN.

By inspecting distributions and summary statistics of **MfgPlaytime**, **ComMinPlaytime**, and **ComMaxPlaytime**, we found that MfgPlaytime was identical to ComMaxPlaytime for all records; therefore, we retained only the community-based variables (ComMin and ComMax) for the analysis. As previously observed, several rows contained logically inconsistent values and we corrected them with these approaches:

- **ComMinPlaytime > ComMaxPlaytime with ComMaxPlaytime $\neq$ 0**: we swapped the two values, restoring the correct ordering without changing the underlying pair.
- **Exactly one value equals 0 and the other is valid ($\neq$ 0)**: we copied the non-zero value into the zero entry, consistent with cases in the official website where only one playtime value is effectively provided.
- **Both values equal 0**: we replaced both with NaN, treating them as genuinely missing (i.e., no meaningful community-sourced playtime information available).

According to the feature description, **ComWeight** and **GameWeight** are expected to range from 1 to 5, an inspection of their distributions revealed two main issues: first, all records with GameWeight = 0 also had ComWeight = 0. This pattern was confirmed by NumWeightVotes, which was equal to 0 for exactly the same 506 rows, indicating that zero values correspond to the absence of votes rather than valid ratings. Since retaining these zeros as numeric values would bias the analysis, all zero entries were replaced with NaN, resulting in 506 missing values.

Second, while GameWeight never exceeded the upper bound of 5, ComWeight contained 13 slightly out-of-range values (5.004–5.2132). Given the very strong agreement between the two attributes (r = 0.997) and the small per-row discrepancy (mean absolute difference 0.19, range 0.0–0.3), we didn't fix them and we drop the column.

The attribute **MfgAgeRec** contained 1325 entries equal to 0, which do not represent valid ages and are used as substitutes when the manufacturer has not provided an age recommendation. We started by using ComAgeRec (when available) to impute missing values, fixing 995 rows (75.09%). This choice was motivated not only by its strong correlation when considering only non-zero entries (=0.72), but more importantly by its direct semantic alignment: both variables refer to recommended age and share the same domain.

To estimate the remaining missing values, we followed an intuition: the recommended age of a game is likely related to its publication family, thematic categories, and overall complexity. We validated this assumption on rows with MfgAge != 0, by deriving three average ages from *games sharing the same Family, the game's active Category indicators, and games with similar GameWeight, defined as those within an absolute window of* ± 0.5. Correlation results (As shown in Table 1) supported the use of these proxies. Before aggregating them for imputation, we also checked the consistency across estimators at the row level by comparing their

resulting distributions and pairwise differences: estimates did not exhibit large discontinuities, which reinforced the robustness of averaging them. For the last still missing rows, we assigned the final value as their row-wise average. All the imputed data were casted into int() to keep semantic dtype consistent (l'età è un intero).

Variable	Correlation values
ComAgeRec	0.723
FamilyAgeRec	0.864
CatAgeRec	0.750
GameWeightAgeRec	0.516

Table 1: Pearson correlation table of attributes with MfgAgeRec

Some attributes were characterized by a prevalence of zero values or empty structures, these features were deemed non-informative, thus were removed: **BestPlayers**: 90.96% of values being zero; **GoodPlayers**: 90.97% of the entries consisted of empty lists; **NumComments**: zero variance, with 100% of the values identical to 0.

2.4.3 Outlier Detection

A small number of anomalous values were observed in **YearPublished**. Although some entries are negative and could initially be interpreted as outliers (e.g. **-3500**), manual inspection confirmed that they are semantically consistent with the game's origin; therefore, they were retained.

In addition, we inspected all records with **MaxPlayers** ≥ 99 (192 rows) to address semantic inconsistencies. The distribution was dominated by repeated "cap" values: (99: 165 rows, 100: 19 rows, 999: 3 rows). Since these values convey the same notion of very large or open-ended player counts, we standardized them by collapsing 100 and 999 into **99**. Conversely, the remaining high values, 120, 127, 163, 200, and 362 (one row each), were individually verified and retained as legitimate player counts.

Finally, we detected a small number of extreme values in the community playtime attributes (**ComMinPlaytime and ComMaxPlaytime**), including a maximum value of **60,000**. Overall, 7 records exceeded 10,000; manual inspection confirmed they were meaningful in context, so no corrective action was applied.

We adopted these choices to preserve the original dataset as cleanly as possible. Although, for subsequent analyses, we standardized the operational range using a **1st-percentile threshold**, applied only when the gap between min and 1% (or between 99% and max) was extremely large (i.e., indicative of extreme-tail outliers rather than natural variability).

Therefore, this clipping was applied only to the previously mentioned columns: YearPublished, setting the lower bound to **1949**, and MaxPlayers, applying an upper threshold of **30**. It was not applied to ComMaxPlaytime, since that attribute was later log-transformed.

2.5 Pairwise correlation

Since there is no universal threshold we based our benchmark on similar work in related literature.

With threshold **0.6** one third of the variance can be explained linearly by the other variable, and it is a compromise between moderate relation<0.4 and redundant correlation>0.8. This choice helps focus on the most meaningful statistical dependencies avoiding unnecessary noise from weaker correlations.

From the matrix shown in Figure 1, we can observe the correlation between each numerical variable in the dataset calculated using the Pearson correlation method using the previously chosen threshold.

We decided to take corrective action on variables showing a pairwise correlation higher than 0.8. This resulted in the removal of **MfgPlaytime** (identical to ComMaxPlaytime), **ComWeight** (highly correlated with GameWeight), **NumUserRatings** and **NumWeightVotes** (strongly correlated with each other and with NumOwned, reflecting the intuitive relationship that purchased games tend to receive ratings). In addition, this analysis motivated a transformation strategy for variables that, while correlated and belonging to the same semantic domain, capture different perspectives of the same concept, such as *ComMaxPlaytime* and *ComMinPlaytime*, *NumWish* and *NumWant*, and *Rank* and *Cat*.

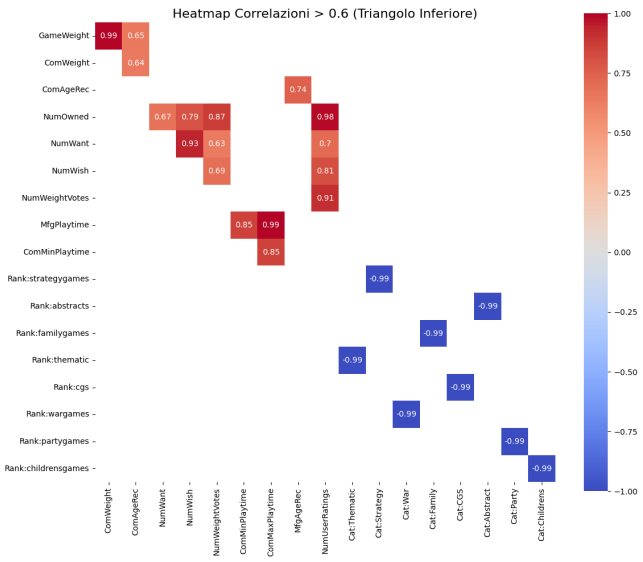


Figure 1: Correlation Matrix Heatmap, Pearson method.

2.6 Data transformation: Feature Engineering and Elimination of Attributes

As highlighted by the correlation analysis, we addressed highly correlated pairs by deriving a single consolidated variable for each pair.

- ComMinPlaytime and ComMaxPlaytime were merged into **AvgPlaytime**, computed as the row-wise mean of the two values.
- NumWish and NumWant were combined into **NumDemand**, computed as the row-wise sum.

Thanks to the correlation, we checked each **Rank–Cat** pair jointly in two stages, showing that for each row every 1 in a given Cat corresponds to a valid Rank value in the same category, while every 0 corresponds to the **placeholder 21926**, with no violations. For this reason, we decided to retain only the Cat columns, already binarized, as they are a convenient way to preserve the game's category information.

However, since in Chapter 4 we will apply several classification techniques on **Rating** (converted in numeric form, low -> 1, medium -> 2, high -> 3, since although it is a categorical variable, it carries an intrinsic ordinal semantics reflecting increasing levels of success), we also wanted to keep a compact signal capturing whether a game is generally ranked high or low. After hypothesizing a link, and confirming it via Pearson correlation (strong negative associations once 21926 is excluded: ranging from -0.822 for Rank:cgs to -0.674 for Rank:strategygames), we created a **Ranking feature**: we normalized each category-specific rank to $[0, 1]$ (because "20 out of 100" is not comparable to "20 out of 1000") and then assigned, per row, the corresponding normalized value when available in a single ranking, or the mean when multiple rankings were present. The correlation between this feature and Rating, considering only the filled rows, is 0.674; since it covered only 49% of the column, we imputed the remaining values with the mean of the observed ones (**0.505**), treating "not ranked" as "average" and minimizing its influence for classification, obtaining an overall correlation of 0.48, the strongest association with Rating after NumDemand.

For the semantic clustering described in Chapter 2, we also constructed a **textual feature** to be embedded by concatenating the game **Name** (after removing punctuation), the **Description** (already lemmatized), and the **Family** attribute, when available.

Several variables (especially count-based and duration features) showed, even after we tried removing outliers, a strongly **right-skewed** distribution in the exploratory analysis. To reduce the dominance of extreme values while preserving the relative ordering of observations, we applied a logarithmic transformation. Since these attributes can legitimately include zeros, we used the log1p transform (**log(1+x)**), which is defined at **x=0** and avoids discarding valid zero counts while behaving similarly to the natural logarithm for larger values.

The log1p transformation was applied to: **AvgPlaytime**, **NumOwned**, **NumDemand**, **NumAlternates**, **NumExpansions**, and **NumImplementations**

Attribute	Drop reason
BestPlayers	Because 90.96% of the values are zero.
ComAgeRec	Due to 5,530 NaNs (25%) and 0.747 correlation with MfgAgeRec, with little distributional difference.
ComMaxPlaytime	Because we retained AvgPlaytime.
ComMinPlaytime	Because we retained AvgPlaytime.
ComWeight	Due to 0.997 correlation with GameWeight.
Description	Because it was merged into the semantic text column.
Family	Due to 15,262 NaNs (70%) and because it was merged into the semantic text column.
GoodPlayers	Because 90.97% of values are empty lists.
ImagePath	Irrelevant for our analysis.
LanguageEase	Due to 5,891 NaNs (27%) and semantic inconsistency.
MfgPlaytime	Because it is identical to MaxPlaytime.
NumComments	Because it is 100% zero.
NumUserRatings	Redundant with NumOwned (correlation 0.985).
NumWant	Converged in NumDemand.
NumWeightVotes	Redundant with NumOwned (correlation 0.875) and highly correlated with NumUserRatings (0.917).
NumWish	Converged in NumDemand.
Rank:*	Because we created the aggregated Ranking feature instead.

Table 2: Eliminated attributes and reasons

Although only **5.52%** of rows (1,210/21,925) contain at least one **NaN**, missingness is clearly clustered: among rows with at least one NaN, 58.8% already have ≥ 2 problematic values, and 23.5% reach ≥ 3 issues, indicating that NaNs rarely occur as isolated gaps. This makes imputation risky, because it would disproportionately act on observations that are already multi-problematic. In contrast, the values we estimated during cleaning (quality flags, previously marked to keep track of our changes) are mostly isolated corrections: 83.25% of flagged rows (1,292/1,552) contain no NaN, and 82.4% of those have exactly one flag. Therefore, dropping NaN rows is a conservative and transparent choice that preserves reliable data and keeps the dataset compatible with downstream analyses.

```
final df: [ 'YearPublished', 'AvgPlaytime', 'GameWeight', 'MfgAgeRec', 'MinPlayers', 'MaxPlayers',
'Ranking', 'Rating', 'SemanticString', 'NumOwned', 'NumDemand', 'NumAlternates', 'NumExpansions',
'NumImplementations', 'IsReimplementation', 'Kickstarted', 'Cat:Thematic', 'Cat:Strategy',
'Cat:War',  'Cat:Family', 'Cat:CGS', 'Cat:Abstract', 'Cat:Party', 'Cat:Childrens' ] 24 x 20715
```

3 Clustering

For clustering analysis we employed three approaches: Centroid-Based, Distance-Based, and Hierarchical Clustering. Then, to extract additional information from the dataset through textual variables, which cannot be directly handled in the same feature space, we applied a semantic clustering pipeline using BERTopic.

Attributes selected

We focused all analyses, to keep the results consistent and comparable, on the following feature set: **YearPublished**, **MfgAgeRec**, **MaxPlayers**, **NumOwned**, **GameWeight**, and **AvgPlaytime**. The last two were both retained because, despite being moderately correlated ($\rho = 0.73$), they describe different domains (complexity vs. duration); we also tested excluding one of them, and the results did not change appreciably. We did not use the whole set of attributes, to avoid sensitivity to the curse of dimensionality, according to these ulterior proofs:

- **NumDemand**, excluded due to its correlation with NumOwned and their semantic similarity;
- **MinPlayers** was excluded due to its semantic redundancy with MaxPlayers and its highly concentrated distribution (MinPlayers = 2 in 68.76% of games, including = 1 and 3, the share rises to 97.44%), while the often large gap between those confirm that MaxP. offers more informative variability for distinguishing games intended for smaller versus larger groups;

- **NumAlternates**, **NumImplementations**, and **NumExpansions**, excluded because of their zero-inflated distributions (59.24%, 77.37%, and 73.45% zeros, respectively);

Furthermore, since categories represent a relevant dimension for the analysis, but their binary encoding could substantially distort the distances used by clustering algorithms, we experimented with their inclusion through weighting in the **standardized feature space**. Specifically, each of the binary *Cat:** was scaled by a factor of $1/\sqrt{k}$ **(**$1/\sqrt{8} = 0.35$**)**, so that the overall contribution of categorical information to the Euclidean distance was comparable to that of a single quantitative feature. Despite this adjustment, the results were **unsatisfactory**: the resulting clusters were predominantly driven by categorical membership, reducing the ability to identify structures governed by the selected numerical variables. For this reason, categorical features were excluded, in order to ensure coherence and comparability across the different clustering methods.

Scaling:

We adopted `MinMaxScaler()` to ensure direct comparability across heterogeneous features, as it consistently produced the most stable and well-separated cluster structures across all three clustering approaches. In our experiments, it outperformed StandardScaler (slightly lower quantitative scores overall and less distinct separation in pairwise 2D projections) and RobustScaler (worse performance, tested on non–log-transformed data).

Evaluation and Visualization Methods:

To compare and interpret the resulting clusterings in a clear, data-driven manner, we combined five complementary evaluation and visualization techniques:

- We employed **boxplots** of the feature distributions to describe cluster composition within each cluster.
- To quickly compare the global consistency of different clusterings at a glance, we adopted **6-dimensional spider plots**: for each cluster, we computed the **mean profile** over the selected features and normalized it to [0,1], so that scales were comparable and the focus remained on the overall shape of the clusters rather than minor boundary shifts (i.e., small reassignments of points) that do not alter the average pattern.
- To quantify which variables discriminated the groups most, we ran a **one-way ANOVA** for each feature and computed the global effect size η^2, measuring the proportion of variance explained by **cluster membership** and providing a robust ranking of the most informative features.
- **A** PCA projection to obtain an overall view of the cluster structure. To systematically inspect how clusters distribute across individual variables, detect unexpected separations, and analyze boundary regions more accurately, we visualized all **2D projections** of the original feature pairs, identifying the subspaces where segmentation is sharpest or where overlaps persist.

Finally, as **quantitative** assessment **metrics**, we relied on the silhouette score to evaluate geometric separation, and on **CV**, the coefficient of variation of cluster sizes (standard deviation divided by the mean), to quantify how balanced the cluster-size distribution is, with higher values indicating stronger size imbalance.

3.1 Analysis by centroid-based methods

3.1.1 Parameter selection and application:

We started our analysis with **K-Means**; to select the number k of clusters, we combined the **elbow method** (based on SSE) with the **silhouette score**. The resulting plot indicates that **k=5** and **k=7** achieve the highest and identical silhouette among the values that lie in the elbow region (**0.220**). Although **k=3** yields the best silhouette overall (**0.297**) and **k=4** is close (**0.224**), the SSE reduction from k=3 to the immediately larger values remains substantial. For completeness, we investigated all four configurations (k=3,4,5,7).

To assess the robustness of these findings, we also applied **Bisecting K-Means**, in its standard configuration (default bisection strategy *biggest_inertia*) in a variant with multiple initializations. Using the same elbow–silhouette approach, the best silhouette was obtained at **k=4** (**0.2540**). However, **k=5** provided the second-best silhouette among values > 4 (**0.1905**) and was more consistent with the elbow trend. We therefore examined both k=4 and k=5, and compared them against the corresponding K-Means results.

3.1.2 Results:

We first report the analysis of K-Means with **k=5**, which we use as a baseline reference to interpret and compare the other solutions.

As also shown in Figure 2, **GameWeight** and **YearPublished**, the two features that primarily drive the clustering according to the η^2 effect size (0.78 and 0.71 respectively), clearly separate the data into five interpretable groups: *old and simple games* (cluster 2, **10.5%** of all games), *old and complex games* (cluster 4, **9.1%** of all games), *new and very simple games* (cluster 0, **30.8%** of all games), *new and moderately complex games* (cluster 3, **33%** of all games), *and new and very complex games* (cluster 1, **16.7%** of all games).

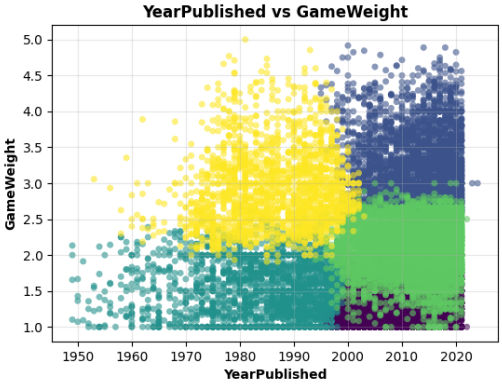


Figure 2: K-Means clustering ($k = 5$)

Given the strong correlation between **GameWeight** and **AvgPlaytime**, which is the third most discriminative feature, it follows a similar, but less sharp, pattern, with simpler games generally associated with shorter durations and more complex games with longer ones. From the remaining 2D projections, we also observe that clusters characterized by simpler games (**2 and 0**) tend to have a lower recommended age, while for the complex cluster vice versa (**1, 4, 3**). Interestingly, the former group (**2, 0**) also exhibits higher variability in MaxPlayers, reaching the largest values observed. In contrast, NumOwned appears approximately uniformly distributed across clusters and plays a negligible role in the segmentation. As we can see this configuration shows signs of boundary instability, particularly affecting clusters 0 and 3, due to the effect of these features.

For k=7, the solution shows partial **overlap** among clusters and mainly results in a further subdivision of two existing clusters (3 and 0) along secondary dimensions. In particular, the first split separates new, moderately complex games into **more** and **less popular titles**, while the other divides new, very simple games according to the maximum number of players (with a threshold around 8-10). Consequently, k=7 is the one, also confirmed by η^2 **values**, in which all dimensions contribute to the clustering.

Although the **K-Means solutions k=3** and **k=4** are geometrically **well separated** in feature space and still rely almost exclusively on YearPublished and GameWeight, they fail to fully respect domain semantics: cluster boundaries emerge as oblique rather than aligned with more natural vertical or horizontal thresholds. Bisecting K-Means confirms and improves upon these configurations. As shown by the spider plots, the **k=5** Bisecting solution pretty closely matches the corresponding K-Means structure, but old games are split across the same complexity thresholds as new ones. Inspection of the 2D projections reveals that the new simple games are now divided with a further refinement, not on a minimal change in complexity, but separating children-oriented titles ($\leq$ 6–7 years) and the remaining ones.

The **k=4 Bisecting K-Means** solution further improves upon standard K-Means, resulting in a configuration that is superior both **geometrically** (higher silhouette) and **semantically**. It merges the last two clusters mentioned above and yields a **clean partition** into old vs. new games (threshold around 2000) and simple vs. more complex games (threshold around 2), covering all four meaningful combinations.

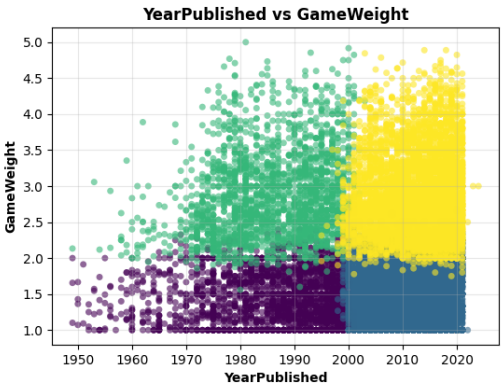


Figure 3: Bisecting K-means k=4

Given the less clearly defined overlaps observed in configurations with larger k, we ultimately selected **Bisecting K-Means** with **k=4** as the best overall solution (shown in Figure 3). This choice is supported both by semantic considerations, clear and consistent thresholds across features, and by geometric quality, as reflected by the higher achieved silhouette score. At the same time, analyzing the alternative configurations proved valuable for understanding how these four robust components are internally subdivided and which characteristics bring them closer together.

3.2 Analysis by density-based methods

3.2.1 Parameter selection and application:

We initially tested **DBSCAN**. This method relies on a single distance metric (**Euclidean** by def.; we also tested **Manhattan**) and requires the configuration of two parameters: **min_samples** and **eps**. As a first exploratory step, we adopted three common heuristics to estimate min_samples: **d+1**, **2d**, and **log(n)**, where d is the dimensionality of the feature space, then the corresponding eps values were estimated using k-nearest-neighbor distance analysis (k-distance method).

All three configurations led to unsatisfactory outcomes, typically resulting in a single dominant cluster with varying amounts of noise, or in the presence of only a few very small micro-clusters.

We therefore proceeded with a **conservative grid search**, still with Euclidean and Manhattan, varying min_samples in the range [30 values from **2-200**] and eps in the range [**0.01-2**] (both with variable step sizes). For each configuration, we recorded the **silhouette score**, **outlier ratio**, number of **points** assigned to the main cluster (excluding noise), and the cluster-size coefficient of variation (**CV**). Despite the wide range of tested configurations, and restricting the analysis to results with more than one cluster and less than 70% noise, the outcomes remained qualitatively unchanged.

To further confirm that density-based clustering was **not suitable** for our dataset, we additionally tested **OPTICS** and **HDBSCAN**. Due to computational constraints, OPTICS was evaluated on a random subset of 5,000 samples. For both methods, we performed grid searches using the same evaluation metrics described above, with the following configurations:

- **OPTICS:** we performed a grid search over the hyperparameters, after further analysis: `min_samples` $\in \{5, 10, 20, 50\}$, `min_cluster_size` $\in \{20, 50, 100, 200\}$, and $xi \in [0.005, 0.05]$ sampled at 8 discrete values (uniform grid).
- **HDBSCAN:** we explored `metric` $\in \{$`euclidean`, `manhattan`$\}$ and `cluster_selection_method` $\in \{$`eom`, `leaf`$\}$. For density/cluster size parameters, we used logarithmically spaced grids: `min_samples` $\in \{2, \ldots, 200\}$ with 20 log-spaced values (plus `min_samples` = `None`), and `min_cluster_size` $\in \{5, \ldots, 1500\}$ with 30 log-spaced values.

3.2.2 Results:

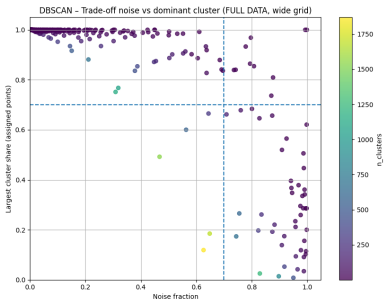


Figure 4: Trade-off between noise fraction and dominant cluster size in DBSCAN.

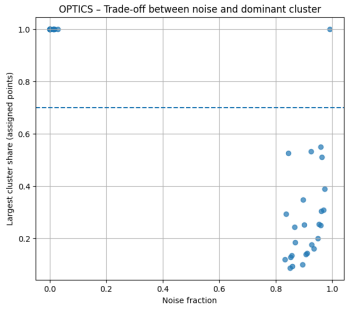


Figure 5: Trade-off between noise fraction and dominant cluster size in OPTICS.

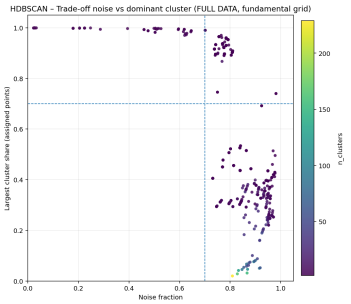


Figure 6: Trade-off between noise fraction and dominant cluster size in HDBSCAN.

The three plots reported here support our conclusion. In the graphs, the percentage of the largest cluster share is computed only with respect to the non-noise points, to keep the analysis coherent. Figures4, 5, 6 show that there are **no configurations** that simultaneously achieve more than one cluster while maintaining both a reasonably balanced cluster-size distribution and an acceptable noise level.

3.3 Analysis by hierarchical clustering

3.3.1 Parameter selection and application:

We began by constructing **hierarchical dendrograms** employing **Euclidean** and **Manhattan** distances as dissimilarity measures and, for each metric, evaluated the **single**, **complete**, and **average** linkage criteria. We applied **Ward's linkage** only under the Euclidean setting, since Ward's method is defined in terms of minimizing within-cluster variance and is therefore compatible only with Euclidean distances.

Dendrograms were computed on all data from the full linkage matrix and visualized in a **truncated form** for readability. Overall, **Ward + Euclidean** emerged as the most robust configuration: the dendrogram shows large vertical jumps in the final merges (i.e., substantial increases in dissimilarity), which typically indicate clear separations between macro-groups and a clean hierarchical structure, consistent with Ward's variance-minimization principle. **Complete linkage** also tends to mitigate chaining effects and produces more compact trees; however, its structure is generally less pronounced than Ward's, and results are more sensitive to the chosen distance metric. In particular, Euclidean distance more clearly reveals macro-clusters, whereas

Manhattan distance leads to greater fragmentation, suggesting finer-grained partitions. In contrast, **average** and especially **single linkage** exhibit the classic **chaining effect**, producing ladder-like and highly asymmetric dendrograms in which a large cluster progressively absorbs observations (or small groups) through many closely spaced merges.

We use **cophenetic correlation coefficient (CCC)** to further supports these observations. The dendrograms that best preserved pairwise distances were those obtained with **average linkage** (Euclidean **0.63**, Manhattan **0.59**), followed by **Ward + Euclidean** (**0.49**), while **complete** and **single linkage** showed lower CCC values.

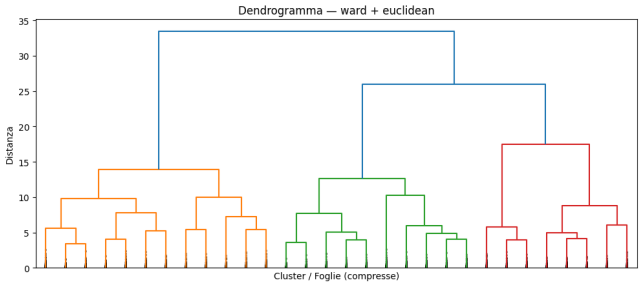


Figure 7: Dendrogram using Ward+Euclidean

To systematically validate these qualitative insights, we performed a **grid search** across all configurations, evaluating values of k from **2 to 10**, with particular attention to candidates suggested by the **largest gap** criterion on the dendrogram. Specifically, we analyzed the heights of the merges in the linkage matrix and selected candidate cuts corresponding to the largest increases in dissimilarity between consecutive merges, interpreted as separations between macro-groups (typically between k=2 and k=5; for Ward, mainly k=3 or k=4).

For each configuration, we recorded the **silhouette score**, **SSE** (for Euclidean-based methods), minimum and maximum **cluster sizes**, and the **coefficient of variation** (CV) of cluster sizes. The results confirmed our initial assessment: **Ward's method** is the only approach that consistently achieves a low CV (indicating well-balanced cluster sizes) across all values of k, while also maintaining a reasonable silhouette score (e.g., k=3:0.2468, k=4:0.2315, k=5:0.1527).

By contrast, single and average linkage (the latter to a lesser extent) yield artificially high silhouette values, for example, **single–Euclidean** achieves 0.5559 at k=3 and remains ≥ 0.3609 up to k=10. However, these scores are inflated by degenerate structures consisting of a **single mega-cluster** and many **micro-clusters** or outliers, making such solutions unreliable and poorly interpretable for segmentation purposes. **Complete linkage** provides acceptable but inferior results compared to **Ward**, underperforming both in silhouette and in cluster-size balance (higher CV values). Across all linkage criteria, **Euclidean** distance consistently yields better average performance than **Manhattan**.

3.3.2 Results:

We therefore first analyze **k=3** and **k=4**, as they represent the best-performing solutions and are also the values suggested by the dendrogram cut. A clear correspondence immediately emerges with the **k=4** Bisecting K-Means solution discussed earlier, both in terms of semantic interpretation and cluster-size distribution. However, from the plot we observe that the hierarchical **k=4** partition is less semantically coherent than the previously selected configuration: it is more sensitive to boundary-point assignments and produces less "linear" separations, resulting in less stable thresholds.

We then also examine **k=5**. Although its silhouette is lower, it lies directly on the elbow and allows a consistent comparison with the K-Means results. Here as well, the solution largely matches the corresponding partitioning outcome, but it is strongly affected by the same boundary instability noted above, similarly to the **k=3** configuration.

3.4 Semantic Clustering

3.4.1 Parameter selection and application:

We transformed the textual feature **SemanticString**, previously constructed by concatenating *Name*, *Description*, and *Family* (when available), into 1024-dimensional embeddings using the Jina-v3 model, which currently represents a state-of-the-art embedding approach for clustering tasks according to the MTEB benchmark. For

clustering the resulting embeddings, we adopted **BERTopic** using its default pipeline, which integrates a dimensionality reduction step (UMAP by default), a clustering algorithm (**HDBSCAN** by default), and a c-TF-IDF representation to extract and characterize the most representative keywords for each cluster.

We initially tested the model using the default parameters, both with and without outlier reduction, we obtained **313** clusters with 7,754 outliers out of 20,714 records (37.43%), achieving a silhouette of **0.094** in the UMAP space and -0.025 when computed directly on the original embeddings. Despite these modest quantitative scores, the clusters were already semantically coherent. To improve silhouette performance and reduce the number of clusters, thus increasing ability, we performed a targeted grid search and this optimization yielded some configurations with an embedding-space silhouette > 0.11 (which we consider more appropriate for semantic clustering, as it evaluates separation in the original space), while keeping the noise ratio below 40% and producing between 50 and 150 clusters (We choose the configuration: `umap_nn` $= 45$, `umap_nc` $= 5$, `umap_md` $= 0.0$, `hdb_mcs` $= 60$, `hdb_ms` $= 10$, `hdb_met` $=$ `"eom"`, con 65 cluster e `sil_emb` $= 0.1047$).

The **c-TF-IDF** representation allows us to easily identify the theme of each cluster from the extracted keywords (es. *[K = 60] [Topic 23] documenti*= 416 *keywords: battle, napoleon, army, war, french, napoleonic, prussian, unit, rule, waterloo, campaign...*).

The ten most frequent game themes are war (nearly 14% of the games), card games, crime, fantasy, dice games, space missions, medieval settings, construction, animal-themed games, and pirates.

To gain clearer control over the number of clusters and avoid explicit outlier labeling (which, although reassignable, is intrinsic in the HDBSCAN-based setup), we tested the same pipeline by replacing the clustering step with **K-Means**. To select a reasonable value of k, we again combined the **elbow method** (SSE) with the 2 **silhouette score**, computed both in the UMAP space and on the original embedding space. In addition, we recorded cv_coherence (*keyword co-occurrence coherence of cluster topics*) and c_semantic (*intra-cluster semantic coherence of c-TF-IDF terms*) over a range of $k \in [10, 200]$ with step 10, using the UMAP-reduced space to remain consistent with the pipeline.

As expected, the silhouette tended to increase with larger **k**, while the semantic coherence metrics showed the opposite trend. This behavior is normal: as k grows, broad macro-topics are split into smaller, more specific clusters, and the **c-TF-IDF** keywords become more specialized, which can reduce global coherence scores. In line with this trade-off and to keep the analysis manageable, we selected k=60 and reported an interactive visualization that we developed to support qualitative inspection.

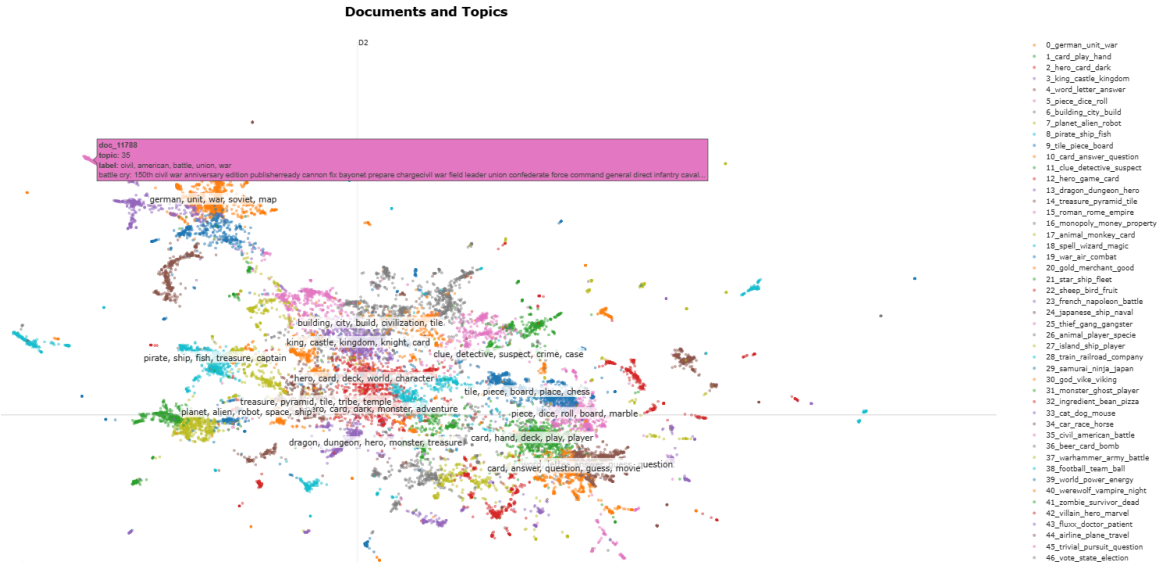


Figure 8: Semantic clustering

3.4.2 Results:

As highlighted in the upper-left macro-cluster of Figure 8, a large thematic group clearly emerges around **war-related games (2,679 titles)**, further structured into coherent sub-themes such as World War scenarios, the American Civil War, and Napoleonic campaigns. Through the interactive graph, we also observe strong internal coherence within individual sub-clusters: despite the 2D reduced representation, games cluster tightly

around very specific topics (e.g., Normandy battles appearing close to one another), enabling a finer-grained qualitative inspection.

Although only briefly summarized here, this semantic analysis provides an immediate, high-level understanding of the **main thematic domains** present in the dataset and of the **relationships between them**, allowing the overall structure of game themes to be grasped at a glance.

3.5 Final Discussion

The comparative analysis highlights a clear differentiation in performance across algorithmic families, that may be instrinsic in the dataset's structure. Rather than restating individual metrics, this discussion focuses on the comparative behavior of the algorithms and the underlying factors driving these results.

Density-based algorithms (DBSCAN, OPTICS, HDBSCAN) consistently failed to identify meaningful segmentations. Across all experiments, the outcomes were dominated either by a single large cluster either with substantial noise, or with a lot of micro-clusters. This behavior suggestion that the data does not exhibit well-defined regions of homogeneous density. We then hypotized that observations are distributed gradually along continuous variables (most notably *YearPublished* and *GameWeight*), making it impossible for these methods to isolate structure from noise effectively.

Hierarchical methods (specifically Ward's linkage with Euclidean distance) showed full agreement, identifying highly consistent and interpretable structures. This convergence might indicate the presence of stable, global partitions driven by a small subset of dominant features. We think Ward's method performed best because its variance-minimization principle is ideally suited for the compact, balanced clusters found in this dataset. Conversely, other linkage criteria were rejected as they produced degenerate solutions or chaining effects. Despite this consistency, hierarchical solutions exhibited greater sensitivity to boundary-point assignments compared to the more stable partitions of Bisecting K-Means (as seen in Figure 9 in the clusters representing old games).

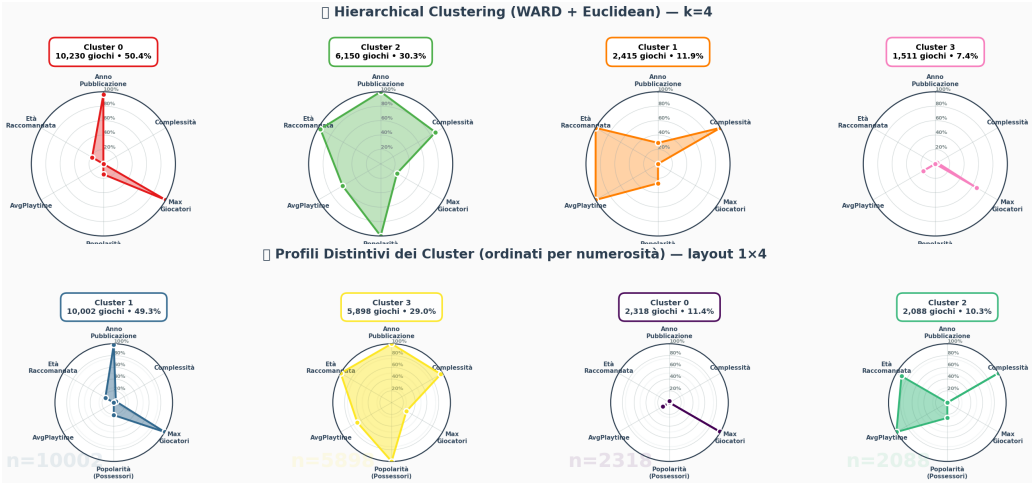


Figure 9: 6-Plots Spider Graphs for Hierarchical-Ward and Bisecting k-Means with k=4

Cetroid-based, with **Bisecting K-Means** with $k = 4$ emerged as the best solution due to its superior balance of stability (Silouhette=0.2540,the best result) and interpretability. It defines a clean four-way partition based on release year (circa 2000) and complexity (*GameWeight*=2), aligning closely with domain semantics. While higher values of k reveal finer sub-structures such as popularity or player count, they primarily over-segment coherent groups augmenting noise points on the boundary.

In conclusion, a **semantic clustering pipeline** (BERTopic) provides a level of granularity that quantitative variables alone cannot reach. While numerical features define the "physical" evolution of games (duration, complexity, age), semantic analysis reveals the thematic domains (such as war-related, medieval, or fantasy settings) that coexist within those structural clusters. Integrating these two perspectives enables a multi-dimensional understanding: numerical features establish the "skeleton" of the market, while semantic topics populate it with specific genres and trends.

4 Classification

We classified the **Rating** variable with a splitted dataset into training (70%) and test (30%) sets using stratified sampling to preserve class proportions. All methods were evaluated using the same set of metrics: confusion matrix, accuracy, precision, recall, F1-score, and ROC curve.

4.1 KNN

4.1.1 Parameter selection and application:

For the K-Nearest Neighbors (KNN) classifier, the best configuration was identified after a feature selection and hyperparameter tuning phase: `MinMaxScaler()` outperformed StandardScaler, and similarly weights="distance" achieved better results than uniform. Regarding the distance measure, the best-performing configuration was metric="manhattan" (after also comparing euclidean, minkowski (p=1.5, halfway between the first two), chebyshev, and canberra).

The selection of the number of neighbors was performed by exploring k in the range 1–50 via cross-validation, obtaining k = 21 as the optimal value. Finally, during model selection, **scoring="f1_macro"** was used as the evaluation criterion because, since the Rating variable is imbalanced, it allows a balanced evaluation of performance across the three target classes, assigning equal weight to each class and reducing the risk that performance is dominated by the most frequent class.

4.1.2 Results:

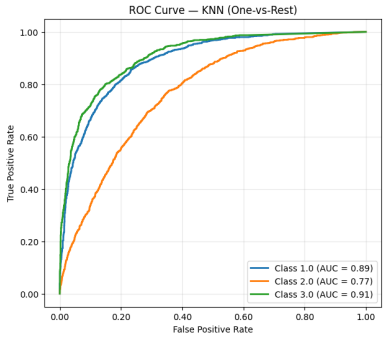


Figure 10: ROC Curve for KNN (One-vs-Rest).

Class of Rating	Precision	Recall	F1-score	Support
1.0	0.75	0.69	0.71	1946
2.0	0.65	0.75	0.69	2724
3.0	0.76	0.62	0.68	1422
Accuracy			0.70	6092
Macro avg	0.72	0.68	0.70	6092
Weighted avg	0.71	0.70	0.70	6092

Table 3: Classification metrics computed for the rating classes (KNN).

1334	596	16
425	2042	257
27	516	879

Confusion matrix (KNN).

The optimized KNN classifier achieved an overall **accuracy** of **0.70** on the test set (Table 3), with a **macro-averaged F1-score** of **0.70**, indicating a balanced performance across the three rating classes. As reported in the Table, **Class 1 (low)** and **Class 3 (high)** show comparable precision values (0.75 and 0.76), while **Class 2 (medium)** exhibits slightly lower precision (0.65) but the highest recall (0.75), suggesting a tendency of the model to favor this intermediate class. Conversely, **Class 3** presents a lower recall (0.62), indicating that a substantial portion of its instances is misclassified, mainly as Class 2.

This behavior is confirmed by the confusion matrix, which highlights that most misclassifications occur between adjacent rating classes, rather than between extreme ones, reflecting a natural overlap in the feature space. The ROC analysis, performed using a One-vs-Rest strategy, further supports these findings: **Class 1** and **Class 3** achieve high discriminative capability with **AUC values of 0.89 and 0.91**, respectively, while **Class 2** shows a lower yet acceptable **AUC of 0.77**, confirming its greater ambiguity. These results indicate that the KNN model is effective in separating low and high ratings, while moderate ratings remain intrinsically harder to distinguish due to overlapping characteristics.

4.2 Naïve Bayes

4.2.1 Parameter selection and application:

For Naive Bayes classification, we tested both **GaussianNB** on continuous attributes (**YearPublished**, **MfgAgeRec**, **MinPlayers**, **MaxPlayers**, **AvgPlaytime**, **GameWeight**, **NumOwned**, **NumDemand**, **NumAlternates**, **NumExpansions**, **NumImplementations**) and **CategoricalNB** on the same ones, discretized.

The first algorithm was scaled using `MinMaxScaler()`, in order to keep the preprocessing consistent with the other classifiers.

For CategoricalNB, we evaluated two distinct discretization approaches: a first, purely **data-driven** approach based on *qcut* (with 8 or 10 bins yielding the best configurations), and a second, more semantic approach, in which variables were discretized through a **hybrid discretization** strategy: domain-driven/manual binning for semantic attributes, with human-defined boundaries that respect the observed distributions, combined with Jenks natural breaks (discussed in more detail in Chapter 5) applied to selected log-transformed count-like attributes. Overall, this second strategy led only to a minimal improvement over the baseline.

During Naive Bayes model selection, we also performed a feature ablation analysis, testing multiple subsets of attributes: the exclusion of AvgPlaytime, NumOwned, and NumAlternates slightly improved the results across all configurations. This is coherent with the nature of the model, since naive bayes would require linear independence and the removed features are the ones slightly redundant. Finally, **YearPublished** emerged as a fundamental feature, as removing it caused a sharp drop in performance.

4.2.2 Results:

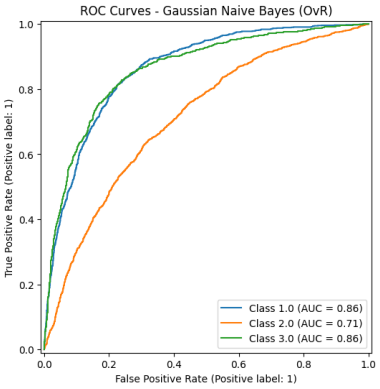


Figure 11: ROC Curves for Gaussian Naive Bayes (One-vs-Rest).

Class of Rating	Precision	Recall	F1-score	Support
1.0	0.63	0.78	0.70	1946
2.0	0.64	0.55	0.59	2724
3.0	0.65	0.61	0.63	1422
Accuracy			0.64	6092
Macro avg	0.64	0.65	0.64	6092
Weighted avg	0.64	0.64	0.64	6092

Table 4: Classification metrics computed for the rating classes (Gaussian Naive Bayes).

1526	405	15
782	1495	447
96	453	873

Confusion matrix (Gaussian Naive Bayes).

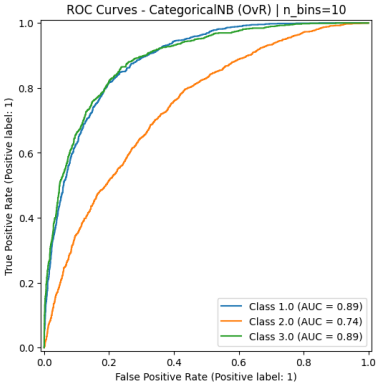


Figure 12: ROC Curves for Categorical Naive Bayes (OvR), $n_{\text{bins}} = 10$.

Class of Rating	Precision	Recall	F1-score	Support
1.0	0.70	0.73	0.72	1946
2.0	0.64	0.64	0.64	2724
3.0	0.67	0.64	0.66	1422
Accuracy			0.67	6092
Macro avg	0.67	0.67	0.67	6092
Weighted avg	0.67	0.67	0.67	6092

Table 5: Classification metrics computed for the rating classes (Categorical Naive Bayes, $n_{\text{bins}} = 10$).

1419	513	14
560	1737	427
42	468	912

Confusion matrix (Categorical Naive Bayes, $n_{bins} = 10$).

As shown in Table 4, the GaussianNB classifier achieves a test accuracy of approximately **0.64**, with a **macro-averaged F1-score of 0.64** and a macro ROC-AUC of about **0.81**. The ROC curves indicate still good discriminative capability for **Classes 1 and 3** (AUC= 0.86), while **Class 2** is consistently the most difficult to separate (AUC= 0.71), as also reflected in the confusion matrix, where most misclassifications occur between adjacent rating classes. Using CategoricalNB with manual discretization combined with Jenks natural breaks (Table 5) leads to a moderate improvement, reaching a test accuracy of approximately **0.67**, a macro F1-score of **0.67**, and a macro ROC-AUC of about **0.84**. The ROC curves confirm a clearer separation for Classes 1 and 3 (AUC=0.89); overall, CategoricalNB provides a consistent but limited gain over GaussianNB, without fundamentally changing the model's error structure.

4.3 Decision Trees

4.3.1 Parameters selection and application:

For the **Decision Tree** model, we first carried out a systematic study of the main hyperparameters to understand how performance changes with model complexity and to identify reasonable value ranges. Specifically, we examined: **criterion** (*gini*, *entropy*), **max_depth** (maximum tree depth), **min_samples_split** (minimum number of samples required to perform a split), and **min_samples_leaf** (minimum number of samples per leaf). We also tracked how training and test performance varied with **max_depth** and the **number of nodes**, in order to select a tree that was expressive but not overly complex, and therefore able to generalize and avoid overfitting (Figure 13)

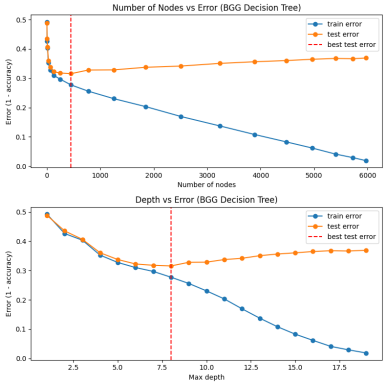


Figure 13: Plot of the training error and validation error vs max depht.

Based on these preliminary analyses, we then performed a broader **validation grid search**, restricted to the most promising hyperparameter ranges, using **cross-validation on the training set** to select the best configuration. The selected model was subsequently evaluated on the **test set**, reporting standard classification metrics. Finally, we applied **cost-complexity pruning** by analyzing the path of effective α values and using cross-validation to assess how pruning affected performance, with the goal of further reducing model complexity without degrading predictive accuracy.

Under the **entropy**impurity criterion, the analysis showed a consistent and stable behavior. We varied the maximum tree depth (**max_depth**) over a wide range and observed that cross-validation performance increases up to intermediate values and then decreases, clearly indicating overfitting for overly deep trees. In particular, **max_depth = 8** provides a good trade-off between expressiveness and generalization (Figure 13).

When analyzing **min_samples_split**, we found that overly small values lead to unstable models, whereas overly large values make the tree excessively rigid; **min_samples_split = 0.01** proved the most effective. Similarly, we evaluated **min_samples_leaf**, that increases up to intermediate values and then declines, with the best configuration around **min_samples_leaf = 30**.

Starting from this setting, we applied **cost-complexity pruning** via **ccp_alpha**, building a sequence of pruned trees and we evaluated them. This step confirmed that the selected configuration lies in a stable region, avoiding both overly complex trees and trivially underfit solutions. We repeated the same analysis with the **GINI** criterion, which achieved slightly lower performance.

We also tested trees generated with **RuleTree**, obtaining similar performance but with a substantially simpler tuning process thanks to its more intuitive controls. The model exposes only a few key hyperparameters (most notably **max_depth** to manage complexity and **prune_useless_leaves=True** to enable automatic pruning of

non-informative branches) making it easier to build readable trees (also supported by its graphical visualization) without implementing a separate pruning procedure.

Including the **Ranking** feature, based on our initial intuition, made a clear difference in this setting: it substantially improved performance and produced the best-performing model across our classification experiments.

4.3.2 Results:

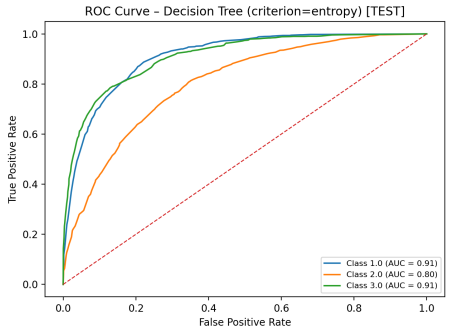


Figure 14: ROC Curve for Decision Tree (criterion=entropy).

Class of Rating	Precision	Recall	F1-score	Support
1.0	0.76	0.72	0.74	1946
2.0	0.67	0.77	0.71	2724
3.0	0.79	0.63	0.70	1422
Accuracy			0.72	6092
Macro avg	0.74	0.70	0.72	6092
Weighted avg	0.72	0.72	0.72	6092

Table 6: Classification metrics computed for the rating classes (Decision Tree, criterion=entropy).

1395	540	11
411	2084	229
31	500	891

Confusion matrix (Decision Tree).

We now present the results for the first selected configuration, based on the **Decision Tree Classifier**, analyzing the outcomes of the final model using the **entropy** impurity criterion in Table 6. The model achieves a test accuracy of approximately **0.72**, with balanced **F1-scores** across the three classes and a **macro-F1 of 0.72**, indicating good generalization and stable behavior even with mildly imbalanced classes. The confusion matrix shows that the remaining errors are mainly concentrated between **adjacent classes**, which is consistent with the ordered nature of the target variable and our previous results.

Cost-complexity pruning further confirmed the robustness of the chosen configuration. The tree was reduced from **219 to 191 nodes** (**−28 nodes, −12.8%**), with **14 leaves removed**, with **no change in test-set performance**. The fact that the maximum depth remained unchanged (**max_depth = 8**) suggests that pruning mainly removed weak, low-information side branches while preserving the core structure, improving model stability and interpretability. This is the expected behavior of an effective cost-complexity pruning procedure: lower complexity with unchanged generalization.

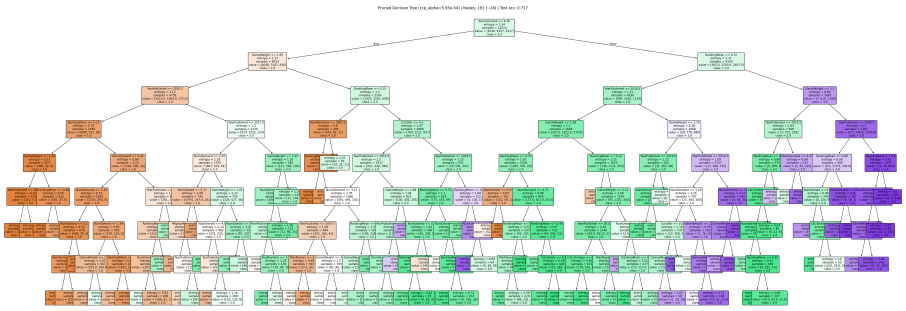


Figure 15: Pruned Decision Tree (ccp_alpha $= 5.95 \times 10^{-4}$). Nodes: 191 (−28). Test accuracy: 0.717.

Using the native feature importance scores, the most influential variables are **NumDemand (0.29)** and **Ranking (0.25)**. Notably, introducing **Ranking** increased test accuracy from **0.68 to 0.72**. This is followed by **YearPublished (0.20)**, a key feature also for Naive Bayes, along with **GameWeight** and **NumOwned**. Thus, we can conclude that the entropy-based Decision Tree with pruning reflects a strong compromise between interpretability, complexity control, and predictive performance, making it one of the most solid and informative models in our classification analysis.

4.4 Final Discussion

A consistent pattern observed across all three methods is the greater difficulty in predicting **class 2 (Medium Rating)**, which systematically emerges as the most ambiguous class. This outcome is not incidental: “medium-

rated" games tend to share overlapping characteristics with both low and high-rated titles, making their separation in the feature space intrinsically more challenging.

KNN effectively separates the extremes but, in regions of uncertainty, tends to favor the intermediate class. **Naïve Bayes** performs worst, likely because its assumptions (conditional independence and simple distributional forms) are poorly suited to our dataset, where several variables, despite belonging to different domains, exhibit correlated substructures and dependencies that the model cannot capture; discretization yields only marginal improvements.

The only model that more clearly mitigates the class Medium ambiguity (while still with greater difficulty than for the other two classes) is the **Decision Tree**, thanks to its ability to model non-linear interactions and explicit feature thresholds. Moreover, its visualization capabilities make it one of the most immediate and interpretable approaches. For these reasons, and given its consistently superior performance across all evaluation metrics, the **Decision Tree–based configuration** represents the most effective and robust choice for our dataset.

5 Regression

In this section, we investigate regression models to quantify relationships among the numerical variables of the dataset. We start with simple regression (one predictor) and then extend to multiple regression (two or more predictors), comparing a linear baseline with non-linear learners to capture possible non-linear effects. In particular, we consider Linear Regression as a reference model and include Decision Tree, Random Forest, and Gradient Boosting regressors to model non-linear patterns that cannot be represented by a single straight line.

Before fixing the final regression setting, we performed an exploratory all-vs-all screening on the main eight numeric variables (**56 ordered pairs**), using a consistent train/test split and evaluating performance through R^2 and MSE. While the strongest numerical signal is obtained for the market/popularity pair (NumDemand → NumOwned, with $R^2 = 0.64$ for Linear Regression and = 0.66-0.67 for non-linear models, best with Gradient Boosting), we chose to focus the subsequent analysis on a relationship that is more informative from a semantic perspective. In particular, we investigate **AvgPlaytime→GameWeight**, as it links two variables that describe different aspects of a board game with different domains, duration and perceived complexity, and therefore allows for a more meaningful interpretation than two closely related popularity indicators.

5.1 Simple Regression

AvgPlaytime shows a high (though not extreme) correlation ($r = 0.72$) with GameWeight, making it a suitable case study to assess both linear and non-linear predictive behavior with simple regression. The goal is to evaluate how well a single predictor can explain the variability in the target variable and to compare a linear baseline against tree-based models that can capture potential non-linear trends.

The results are displayed in Table 7.

Models	**X**	**Y**	**Parameters**	**Coeff**	**Interc**	R^2	**MSE**
Linear Regression	AvgPlaytime	GameWeight	–	0.646020	-0.446534	0.510415	0.316116
Decision Tree	AvgPlaytime	GameWeight	max_depth=7; min_samples_leaf=50; min_samples_split=100	–	–	0.548846	0.296790
Random Forest	AvgPlaytime	GameWeight	n_estimators=300; max_depth=None; min_samples_leaf=50; min_samples_split=100	–	–	0.548855	0.296784
Gradient Boosting	AvgPlaytime	GameWeight	n_estimators=300; lr=0.05; max_depth=3; min_samples_leaf=50; subsample=0.8	–	–	0.548750	0.296852

Table 7: Simple regression results for AvgPlaytime → GameWeight (train/test split).

Overall, the best-performing model is the **Random Forest**, although the three tree-based approaches are extremely close. The top score ($R^2 = 0.549$) means that the model explains about 54.9% of the variability in GameWeight using AvgPlaytime alone, indicating a strong and meaningful relationship between gameplay duration and perceived complexity. **Linear Regression** already achieves a solid baseline ($R^2 = 0.510$, i.e., about 51.0% explained variability), while the consistent improvement of the non-linear models suggests that the association is not purely linear and that tree-based learners can capture additional structure beyond a straight-line trend.

5.2 Multiple Regression

In multiple regression, the target variable remains the same (GameWeight), but the models are trained using multiple predictors in order to improve the quality of the predictions. Since AvgPlaytime was identified as the strongest single-feature predictor in the simple regression setting, we explicitly evaluate its contribution by training each model in two configurations: **WITH AvgPlaytime** included among the predictors, and **WITHOUT AvgPlaytime**, keeping the remaining predictors unchanged. In both settings we use the same predictor set, namely: YearPublished, MfgAgeRec, MinPlayers, MaxPlayers, NumOwned, NumDemand (plus AvgPlaytime in the first configuration).

During linear regression training, multicollinearity was assessed by computing the Variance Inflation Factor (**VIF**) for the predictors. In both configurations, all variables show VIF values well below 10, indicating that it does not compromise the interpretation of the regression coefficients. The highest VIF values are observed for NumDemand (3.541650-3.435442) and NumOwned (3.072027-3.043035), reflecting their partial redundancy, but the magnitude remains moderate.

Models	**Setting**	**Y**	**Parameters**	R^2	**MSE**
Linear Regression	WITH	GameWeight	–	0.628686	0.244267
	WITHOUT		–	0.427092	0.376885
Decision Tree	WITH	GameWeight	Standard settings, as before.	0.653114	0.228197
	WITHOUT		–	0.474887	0.345443
Random Forest	WITH	GameWeight	Standard settings, as before.	0.673921	0.214510
	WITHOUT		–	0.508320	0.323449
Gradient Boosting	WITH	GameWeight	Standard settings, as before.	0.685623	0.206811
	WITHOUT		–	0.526426	0.311538

Table 8: Multiple regression results for GameWeight (WITH vs WITHOUT AvgPlaytime).

As expected, the results improve substantially when AvgPlaytime is included. In the WITH AvgPlaytime setting, the best model is **Gradient Boosting**, achieving $R^2 = 0.6856$ (i.e., explaining about 68.6% of the variability in GameWeight) with MSE = 0.2068. **Random Forest** and **Decision Tree** follow with $R^2 = 0.6739$ and 0.6531, while **Linear Regression** reaches $R^2 = 0.6287$, confirming that a large portion of the relationship is already captured linearly but that non-linear models provide an additional gain.

Removing AvgPlaytime leads to a clear performance drop across all algorithms. In the WITHOUT AvgPlaytime configuration, the best model (again **Gradient Boosting**) decreases to $R^2 = 0.5264$ (52.6% explained variability) with MSE = 0.3115, and Linear Regression falls to $R^2 = 0.4271$.

Overall, the comparison between the two settings quantifies the strong predictive role of **AvgPlaytime**: including it increases explained variability by roughly **+15.9** percentage points for Gradient Boosting (from 52.6% to 68.6%) and by about **+20.2** percentage points for Linear Regression (from 42.7% to 62.9%). This confirms that gameplay duration is a key driver for predicting perceived complexity, while the remaining predictors alone capture only a limited portion of the signal. When instead AvgPlaytime is omitted, NumDemand and MfgAgeRec are the most informative secondary predictors of GameWeight. Both variables become the main surrogates (especially MfgAgeRec in tree-based models), but the overall performance remains substantially lower, indicating that their signal cannot fully substitute playtime.

5.3 Final discussion

Across the regression experiments, non-linear models consistently adapted better to the data than the linear baseline. In the simple regression setting, tree-based methods slightly outperform Linear Regression, suggesting that the relationship between duration and perceived complexity is meaningful and only partially linear. Overall, while Linear Regression provides a strong baseline, the best trade-off in predictive performance is obtained with Gradient Boosting in the multiple-regression setting, and the results support a clear domain-level insight: AveragePlaytime, linearly associated (Pearson), is the key driver in predicting perceived game complexity (GameWeight), but the remaining metadata features provide consistent additional explanatory power, with MfgAgeRec, semantically associated, showing a particularly clear contribution.

6 Pattern Mining

For the preprocessing part of Pattern Mining, an exclusively quantile-based discretization strategy was considered insufficient: despite producing balanced bins, it often lacks semantic meaning and reduces the interpretability of extracted patterns. Moreover this approach can split semantically similar values across different bins while grouping distant values within the same bin, further obscuring meaningful thresholds.

A **semantic discretization** approach was preferred whenever possible. Thresholds and intervals were defined so as to preserve a **logical and domain-consistent** meaning, ensuring that each category corresponds to a conceptually meaningful group, while still remaining coherent with the data distributions and to maintain reasonably balanced bins, avoiding extreme sparsity.

Different discretization techniques were adopted according to the nature of the attributes requiring it and the resulting values were converted into easily interpretable string labels..

- **YearPublished** was discretized in only 4 bins (< 2000, 2000s, 2010-2015, >2015), as a compromise between semantic coherence and frequency distribution. Similarly, **GameWeight**, originally defined on a continuous (decimal) scale, was discretized by rounding to the nearest integer, with the merge of 4 and 5. **MfgAgeRec** was discretized into four age ranges aligned with real standard social age groups.
- **MinPlayers**, due to its distribution (= 2 accounting for more than 68%), was included only when equal to 1, representing games that are also playable solo; **MaxPlayers** discretized by grouping values around the most frequent player counts, with the additional distinction of games playable only-solo when equal to 1.
- For attributes not tied to specific absolute values (**AvgPlaytime**, **NumOwned**, **NumDemand**), but rather to the relative relationships among games (e.g., more or less popular, longer or shorter), we discretized them into three states (Low, Medium, High) using a data-driven approach based on the Jenks natural breaks method: the Jenks library enables the identification of natural breakpoints in the data by minimizing within-class variance and maximizing between-class differences, thereby producing discretization intervals that better reflect the underlying structure of the distribution.
- **Ranking** was also divided into three bins using Jenks, after removing originally unranked games, in order to fully preserve the original semantics of the attribute.
- Attributes representing counts of related entities, such as **NumExpansions**, **NumAlternates** or **NumImplementations** (NumImplementations was included only when IsReimplementation = 0, after verifying that IsImp -> NumImp always holds (conf = 1.0), whereas NumImp -> IsImp doesn't (conf = 0.53), in order to distinguish original games with reimplementations from their reimplementations), because of the extreme prevalence of zero values, were binarized (and included only when true), as the presence or absence of such elements is often more informative than their exact number.
- Also boolean attributes were included only when **true**, in order to reduce sparsity and avoid generating trivial patterns involving the absence of a property.

Finally, each game was represented as a transaction consisting of all its associated items, producing the transactional dataset used for subsequent frequent pattern and association rule mining. After multiple trials with both algorithms, given its efficiency, **FP-Growth** was selected as the preferred over Apriori, as it offers significantly faster processing times.

6.1 Frequent pattern extraction

Frequent pattern extraction was carried out using the FP-Growth algorithm by varying the minimum support threshold (`min_sup`) from 1% to 20% and the minimum itemset size (`z_min`) from 1 to 8 (max itemset length) using a grid-search approach. For each configuration, frequent, closed and maximal itemsets were extracted and compared in terms of cardinality, in order to inspect trends and sensitivity with respect to `min_sup` and `z_min`.

At minimum `min_sup` values, the number of frequent itemsets is extremely **high**. For instance, with `min_sup` = 1% and `z_min` = 2, more than **18,000** frequent itemsets were extracted. The support distribution of the resulting itemsets shows that 70.3% of them have support between 1% and 2%, 24.8% between 2% and 5%, 4% between 5% and 10%, and the remaining 1% spans higher support values, up to a maximum of **28.976657%**, corresponding to **low-demand** and **low-ownership** games.

We therefore observe that by setting the support threshold to just 2%, the number of extracted patterns drops sharply. Similarly, keeping the support at the canonical value of 1% but increasing `z_min`, we obtain 17,312

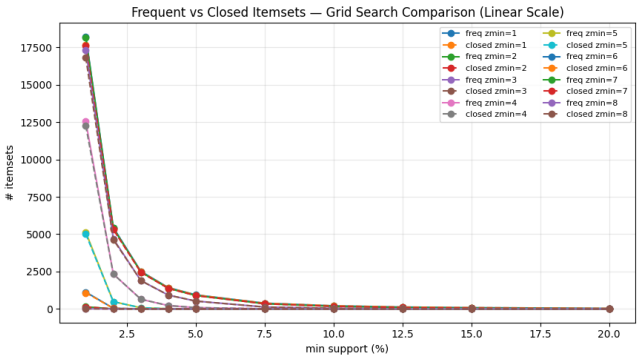


Figure 16: Grid Search Comparison of Frequent vs Closed Itemsets

itemsets for `z_min` = 3, 12,541 for `z_min` = 4, 5,129 for `z_min` = 5, 1,125 for `z_min` = 6, 142 for `z_min` = 7, and finally only 8 itemsets for `z_min` = 8.

In the same way also the closed itemsets (with `min_sup` = 1% and `z_min` = 2 → 17,636) and especially the maximal itemsets (same configuration → 6,183) drops exponentially.

The comparison between frequent and closed itemsets reveals a **strong correspondence** across all configurations. Even at the lowest support threshold (`min_sup` = 1%), the number of closed itemsets is only slightly lower than that of frequent ones, with differences limited to a few percent and as `min_sup` increases, this gap rapidly disappears, indicating that frequent itemsets in the considered dataset are already largely non-redundant and that the additional compression provided by closed itemsets is minimal.

We iterated over `z_min` and inspected the most frequent, interesting, or unexpected patterns, comparing their support against the underlying feature distributions.

A clear and recurring structure emerged across all `z_min` values: the dominant patterns consistently reflect the **low/medium/high** segmentation. As already observed in the previous analyses (and also supported by strong correlations) **rating, ranking, purchase intent, and ownership** are very often aligned within the same tier, with **complexity** frequently reinforcing this coherence (e.g., *GameWeight_1*, *Rating_Low*, *Demand_Low*, *Owned_Low*, *Playtime_Low* at 7.1% support). However, **AvgPlaytime** should be interpreted more cautiously: while it aligns well for the low tier, it often diverges in the other ranges, with high playtime frequently co-occurring with **medium ratings**.

We report below 3 representative and recurring patterns we obtained by removing some columns, to reduce the redundancy we previously discussed:

- `Playtime_High, CatWar, MaxPlayers_2, AgeRec_11_13`, supp=5.604255%;
- `AgeRec_<8, GameWeight_1, MaxPlayers_3_4, Playtime_Low`, supp=4.161332%;
- `MaxPlayers_5_6, Playtime_Med, AgeRec_8_10, GameWeight_2`, supp=5.210283%.

6.2 Association Rules

We therefore generated the rules; as an exploratory analysis we set all parameters to **1**, with an additional filter of 1 also on the support of the rules themselves, plotting histograms showing the number of rules per resulting support range, conf, and lift. It is important to note that we observe promising **confidence** values, with more than 20% of all possible rules having **Conf >= 70**, but low support, with 74.1% between 1 and 2. Taking Conf = 70 as the minimum threshold, we observe that lift ranges from a minimum of 1.5 to a maximum of 19, with a mean value of 3, suggesting further discriminative power, but with almost 80% of rules having support between 1 and 2, and 13.4% between 2 and 3, with only 1.64% above 5, making it difficult to maintain good coverage if only the top n best rules were to be applied.

Moreover, for **each feature** (and subsequently for each specific value), by varying the parameters we analyzed the distribution of the confidence values of the rules in which it appears as a **consequent**, computing descriptive statistics (number of rules, mean, and main quantiles), with the aim of assessing which attributes were associated with rules that are on average more reliable and therefore, in an exploratory sense, more "predictable" within the discretized pattern space (among the most relevant results, **NumDemand** stands out, with more than 23% of total rules exhibiting conf > 0.9, followed **by Playtime** and the **CatWar**). In addition, we implemented a

procedure that, for each feature, and each of its possible target values, prints the highest-lift rules, enabling us to analyze informativeness trends and the associated threshold values.

We report in Table 9 the example of AvgPlaytime, filtered by a rule support greater than 3.

Class	**Antecedent**	**Confidence**	**Lift**	**Support %**
Playtime_High	(CatWar, old_games, GameWeight_3)	0.8538	5.1094	3.3636
Playtime_Low	(AgeRec_<8 GameWeight_1, Rating_Med)	0.9648	2.2723	3.2404
Playtime_Med	(CatStrategy, GameWeight_3)	0.7807	1.9120	3.8560

Table 9: Most representative association rules for the Playtime target (best by lift).

6.3 Rule Based Classifier

Given that we already had a complete rule-extraction pipeline, and comparison thresholds consistent with the other methods applied to the same dataset, we tested whether the Rating target could be predicted using a simple **rule-based classifier**, restricting the rule set to those having **Rating as the consequent**.

We therefore reran the same pipeline after splitting the dataset into a training and a test set. Rules were mined only on the training set, while classification and evaluation were performed on the test set, to avoid bias and overfitting. Every instance was represented as the set of discretized items and classified using a first-match decision list: the predicted class corresponds to the consequent of the first rule whose antecedent is a subset of the instance items; if no rule matches, the majority class is assigned.

We also compared different rule-ordering criteria and found that sorting rules by **descending confidence** consistently outperformed lift-based ordering, suggesting that conditional reliability is more informative than deviation from statistical independence alone. Performance was finally evaluated on the test set using **accuracy**, **confusion matrix**, **per-class metrics**, and **coverage** (Coverage in our model: 0,831, 5062/6092), defined as the fraction of instances matched by at least one rule.

Class	**Precision**	**Recall**	**F1-score**	**Support**
Low (1)	0.73	0.67	0.70	1946
Med (2)	0.64	0.69	0.66	2724
High (3)	0.69	0.67	0.68	1422
accuracy			0.68	6092
macro avg	0.69	0.68	0.68	6092
weighted avg	0.68	0.68	0.68	6092

Table 10: Classification of the Rating variable.

1317	612	17
443	1878	403
37	436	949

Table 11: Confusion Matrix (Low/Med/High Rating).

Our hypotheses were well founded: **coverage** was the main issue. We observed a "linear" improvement in performance as the number of included rules increased by lowering the support thresholds (down to 0.05). Given their low support, when filtered by an adequate confidence threshold (80), although they are ranked high, such rules are rarely applied and therefore do not compromise precision, instead leading to consistently better results in terms of coverage.